

**VIETNAM NATIONAL UNIVERSITY, HO CHI
MINH CITY**

**UNIVERSITY OF INFORMATION
TECHNOLOGY**

FACULTY OF COMPUTER SCIENCE



COURSE

**CS317.Q22 – DEVELOPMENT AND OPERATION OF MACHINE
LEARNING SYSTEMS**

FINAL PROJECT REPORT

**TryOps: Enterprise MLOps for Governed
Virtual Try-On and LLM Serving**

Repository: github.com/trungdangtapcode/TryOps-TryON-MLOps

Instructor: MSc. Do Van Tien

Teaching Assistant: Le Tran Trong Khiem

Students:	Nguyen Tran Nhat Trung	23521684
	Hoang Minh Thai	23521414
	Pham Hai Dang	23520233

Ho Chi Minh City, July 2026

Executive Summary

TryOps is an enterprise-oriented MLOps platform built around two demanding AI workloads: Virtual Try-On (VTON) and Large Language Model (LLM) serving. The project focuses on the operational system around model changes: data and artifact tracking, configured inference endpoints, account and quota control, asynchronous jobs, policy gates, model registry records, observability, incident response, rollback posture, and a clear line between production behavior and test-only diagnostics.

The system is composed of four main layers. The product layer is a React/Vite console that supports login, account/workspace context, LLM requests, VTON upload and job tracking, model registry, governance, dashboards, and history. The edge layer is a Rust Axum gateway responsible for authentication, proxying, request limits, quota admission, guardrail dispatch, static serving, and Prometheus metrics. The workflow layer is a FastAPI backend that owns product APIs, VTON job orchestration, artifact metadata, evaluation, history, account state, and governance views. The inference and control-plane layer includes a host-side FASHN VTON router and workers, an OpenAI-compatible LLM endpoint, Go services, and C++ native tools for policy, metrics, SLO checks, supply-chain records, cache, energy, and evaluation.

The strongest engineering decision in TryOps is the fail-closed model-serving boundary. Production-local defaults require configured VTON and LLM endpoints. If those endpoints are unavailable, the system returns explicit operational errors. This keeps production behavior auditable and separates end-user inference from contract tests.

The current implementation supports a developer-local product stack with Postgres, Valkey, MinIO, MLflow, Keycloak, FastAPI, Rust Gateway, Go guardrail, Prometheus, Grafana, Loki, Tempo, OpenTelemetry Collector, and a host-side FASHN VTON router. The report includes product screenshots for the UI, workspace quota, running VTON jobs, Keycloak, MinIO, MLflow, Grafana, log drilldown, and architecture diagrams.

Keywords: MLOps, Virtual Try-On, FASHN VTON, LLM Serving, vLLM, OpenAI-compatible API, Rust Gateway, FastAPI, Go, C++, Prometheus, Grafana, Loki, MLflow, MinIO, Keycloak, governance, policy gates.

Contents

Executive Summary	i
1 Introduction	1
1.1 Motivation	1
1.2 Project Objectives	1
1.3 Scope	2
1.4 Main Contributions	2
1.5 Technology Stack	3
2 Background and Engineering Principles	4
2.1 MLOps Requirements	4
2.2 Virtual Try-On	4
2.3 LLM Serving	5
3 Requirements Analysis	6
3.1 Actors	6
3.2 Functional Requirements	6
3.3 Non-Functional Requirements	7
3.4 Production vs Diagnostics Boundary	7
4 System Architecture	8
4.1 Architecture Overview	8
4.2 Layered Architecture	10
4.3 Frontend Layer	10
4.4 Gateway Layer	12
4.5 API and Workflow Layer	12
4.6 Data and Artifact Layer	13

4.7	Observability Layer	14
5	Model Serving Design	16
5.1	Production VTON Serving	16
5.2	VTON Job Lifecycle	17
5.3	Multi-GPU Worker Configuration	18
5.4	Production LLM Serving	18
6	Governance, Security, and Responsible AI	19
6.1	Authentication and Workspace Model	19
6.2	Quota and Admission Control	19
6.3	Guardrails	20
6.4	Promotion Gate	20
6.5	Supply Chain and Privacy	21
7	Observability and Operations	22
7.1	Dashboards	22
7.2	Logs	22
7.3	Metrics	22
7.4	Developer-Local Ports	23
7.5	Deployment Preconditions	26
8	Experiments and Evaluation	27
8.1	VTON Benchmark Results	27
8.2	LLM Optimization Results	28
8.3	Native Boundary Evaluation	29
8.4	Balancer Testing	29
9	Limitations and Future Work	30
9.1	Current Limitations	30
9.2	Future Work	31
9.3	Implementation Boundary	31
10	Conclusion	33
	References	34

List of Figures

1.1	TryOps technology stack grouped by product layer.	3
4.1	Product boundary architecture diagram.	8
4.2	Request flow from console entry point to model output and audit records. .	8
4.3	Recommended port surface for shared hosts.	9
4.4	Observability, reliability, and governance summary.	9
4.5	TryOps layered architecture.	10
4.6	TryOps Studio product surface	11
4.7	Workspace and quota view	11
4.8	Running VTON job in the product UI	12
4.9	MinIO artifact store	13
4.10	MLflow benchmark tracking	14
4.11	Grafana observability dashboard	15
4.12	Log drilldown dashboard for debugging jobs and requests	15
5.1	FASHN VTON v1.5 model flow	16
6.1	Keycloak session control and identity surface	19
7.1	Developer-local service and port topology.	24
8.1	VTON benchmark interpretation and average latency comparison.	28
9.1	VTON over-quantization limitation	31

1. Introduction

1.1 Motivation

Many ML applications can produce one impressive screenshot. Production ML systems require much more. A production-grade ML application must answer operational questions that are invisible in a notebook:

- Which data, model weights, code version, configuration, and hardware produced an output?
- Can a bad candidate model be blocked before users see it?
- Does the system report an operational error when a configured model endpoint is unavailable?
- Can each workspace be controlled by quota, rate limits, and concurrency limits?
- Can operators debug failures through metrics, logs, traces, job IDs, and request IDs?
- Can a release be rolled back and explained to a risk reviewer?

TryOps was designed to answer these questions. The chosen workloads are deliberately different. VTON is visual, GPU-heavy, privacy-sensitive, artifact-oriented, and hard to evaluate because visual defects are often qualitative. LLM serving is latency-sensitive, token-cost-sensitive, guardrail-sensitive, and vulnerable to prompt injection and output policy issues. A platform that can operate both workloads behind the same governance spine demonstrates MLOps thinking beyond a single workload.

1.2 Project Objectives

The project objective is to build a working MLOps platform that can serve production model workloads while preserving traceability, observability, quota, and governance.

Concrete objectives:

1. Build a TryOps Console for end users, operators, and reviewers.
2. Use a Rust gateway as the production-facing edge boundary.
3. Use FastAPI as the product workflow backend and model adapter layer.

4. Run VTON through a host-side FASHN VTON serving path.
5. Support LLM inference through an OpenAI-compatible endpoint, including self-hosted vLLM or a managed API.
6. Integrate Keycloak for OIDC login and workspace identity.
7. Integrate Postgres, Valkey, MinIO, and MLflow for product data, counters, objects, and experiment tracking.
8. Integrate Prometheus, Grafana, Loki, Tempo, and OpenTelemetry for observability.
9. Implement model governance through policy gates, supply-chain records, model cards, data cards, and promotion records.
10. Keep test and evaluation utilities separate from production model behavior.

1.3 Scope

Area	Included in the project	Scoped or future work
Product	TryOps Console, LLM Playground, VTON Studio, Account, Registry, Governance, Dashboard, History	Additional accessibility polish, export workflows, and more operator click-through actions
VTON	FASHN VTON router, GPU workers, image upload, async jobs, artifact output, metrics	Larger human preference study and broader production model zoo
LLM	OpenAI-compatible endpoint, OpenAI/vLLM configuration, guardrails, quota	Full live vLLM benchmark depends on the active workspace and GPU availability
MLOps	MLflow, MinIO, promotion records, model/data cards, policy gates	Full production KServe/Kubeflow deployment profile
Observability	Prometheus, Grafana, Loki, Tempo, OTel bridge, structured logs	Complete GPU/process exporters for every shared-datacenter variant
Security	Keycloak, gateway auth, quota, guardrails, supply-chain records	External Secrets/Vault production synchronization
Networking	Developer-local port map and shared-datacenter plan	Final ingress/hardening profile for a multi-tenant data center

1.4 Main Contributions

- A repeatable local deployment path for the full product stack.

- A native-first production boundary: Rust at the edge, Go for services/controllers/load tools, C++ for native tools, and Python for ML workflows.
- A VTON serving abstraction: the API calls a stable serving endpoint, while the router hides worker count, GPU mapping, sockets, and per-worker metrics.
- An LLM adapter using an OpenAI-compatible API and explicit endpoint-error handling.
- A configured observability stack with Grafana dashboards, Loki logs, Tempo traces, Prometheus metrics, and structured job/request logs.
- A governance pipeline containing policy gates, supply-chain records, dependency locks, model scans, provenance, and promotion lifecycle records.

1.5 Technology Stack

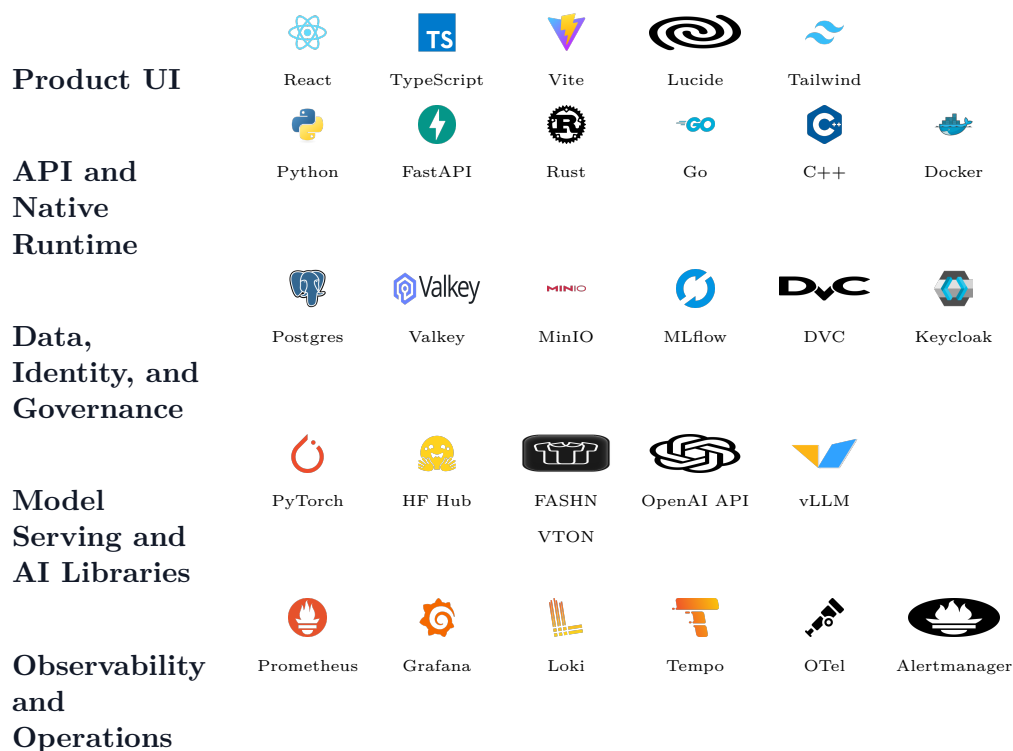


Figure 1.1: TryOps technology stack grouped by product layer.

2. Background and Engineering Principles

2.1 MLOps Requirements

MLOps is the discipline of moving machine learning from research to controlled operation. For this project, the important MLOps requirements are:

- Versioning for data, code, configuration, and model artifacts.
- Experiment tracking and artifact storage.
- Automated evaluation before promotion.
- Model registry with lifecycle states: candidate, challenger, champion, rejected, archived.
- CI/CD and policy gates.
- Observability for metrics, logs, traces, drift, quality, and cost.
- Security, quota, guardrails, and rollback.

TryOps implements these through MLflow, MinIO, Postgres, Valkey, local artifacts, generated reports, repeatable operational targets, native policy/evaluation tools, and Grafana/Loki/Tempo dashboards.

2.2 Virtual Try-On

Virtual Try-On accepts a person image and a garment image, then generates an image of the person wearing that garment. It is operationally difficult because:

- The output is visual, and failures may be geometric distortion, identity drift, texture loss, incorrect pose, broken limbs, or artifacts at garment boundaries.
- GPU memory and latency can be large.
- Person images are privacy-sensitive.
- The system must preserve input, output, report, metrics, and lineage.

- The system must record whether an output came from the production VTON path or from evaluation code.

TryOps uses FASHN VTON v1.5 as the production-local VTON path. The public FASHN research note describes v1.5 as a maskless pixel-space virtual try-on model designed to preserve garment detail without requiring segmentation masks.¹ Evaluation image utilities may be kept for contract tests, but they are not enabled as the default end-user product path.

2.3 LLM Serving

LLM serving requires tracking latency, throughput, token counts, token cost, quota, cache effects, guardrail outcomes, and endpoint failures. TryOps standardizes LLM serving through an OpenAI-compatible API. The endpoint may be self-hosted or managed by a provider, but the production contract remains the same: user prompts are sent to a configured model endpoint, provider secrets are injected through secure runtime configuration, and endpoint failures are surfaced through the product and observability stack.

¹<https://fashn.ai/research/vton-1-5>

3. Requirements Analysis

3.1 Actors

Actor	Needs
End user	Login, upload images, run try-on, inspect status, open results
ML engineer	Run benchmark jobs, compare candidates, inspect generated evaluation reports
Platform engineer	Operate the stack, inspect logs/metrics/traces, manage model endpoints and ports
Risk reviewer	Inspect model cards, data cards, policy gates, supply-chain records, and lineage
Admin/operator	Monitor quota, accounts, registry state, incidents, and roll-back records
Evaluator/instructor	Verify system behavior through code, commands, dashboards, artifacts, and reports

3.2 Functional Requirements

1. Authenticate users with Keycloak/OIDC and bootstrap a workspace.
2. Upload person and garment images.
3. Submit VTON jobs asynchronously.
4. Poll job status and open output artifacts.
5. Send LLM prompts to a configured endpoint.
6. Apply workspace quota and VTON concurrency limits.
7. Store request history, job history, artifact metadata, and feedback.
8. Provide dashboard rollups for operators.
9. Provide governance and model registry views.
10. Support promotion records, rejection records, and rollback records.

11. Export logs, metrics, and traces for debugging.

3.3 Non-Functional Requirements

Requirement	TryOps design response
Configured model execution	Production model endpoints are required for end-user workflows
Fail-closed model serving	LLM and VTON adapters return errors if configured endpoints fail
Reproducibility	Make targets, artifacts, run context, dependency locks, and generated reports
Security	Keycloak, Rust gateway auth, development authentication shortcuts disabled by default
Privacy	Structured logs avoid raw prompt/image content and use job/request IDs
Observability	Prometheus, Grafana, Loki, Tempo, OpenTelemetry, structured logs
Scalability	FASHN router hides multiple GPU workers behind one endpoint
Shared datacenter fit	Plan to expose only necessary ports and keep internal services private
Operability	Repeatable startup/shutdown workflows, smoke tests, health checks, dashboards

3.4 Production vs Diagnostics Boundary

Component	Production path	Test/evaluation utilities
VTON	FASHN router through the configured VTON serving endpoint	Overlay/image utilities only when explicitly enabled for tests
LLM	OpenAI-compatible model endpoint	Request/response harnesses only for adapter checks
Semantic cache	Controlled and observable feature	Disabled by default in production-safety config
API keys	OIDC/JWT and provider secrets	Static development key disabled by default
Worker ports	Hidden behind router	Direct worker port only for debugging

4. System Architecture

4.1 Architecture Overview

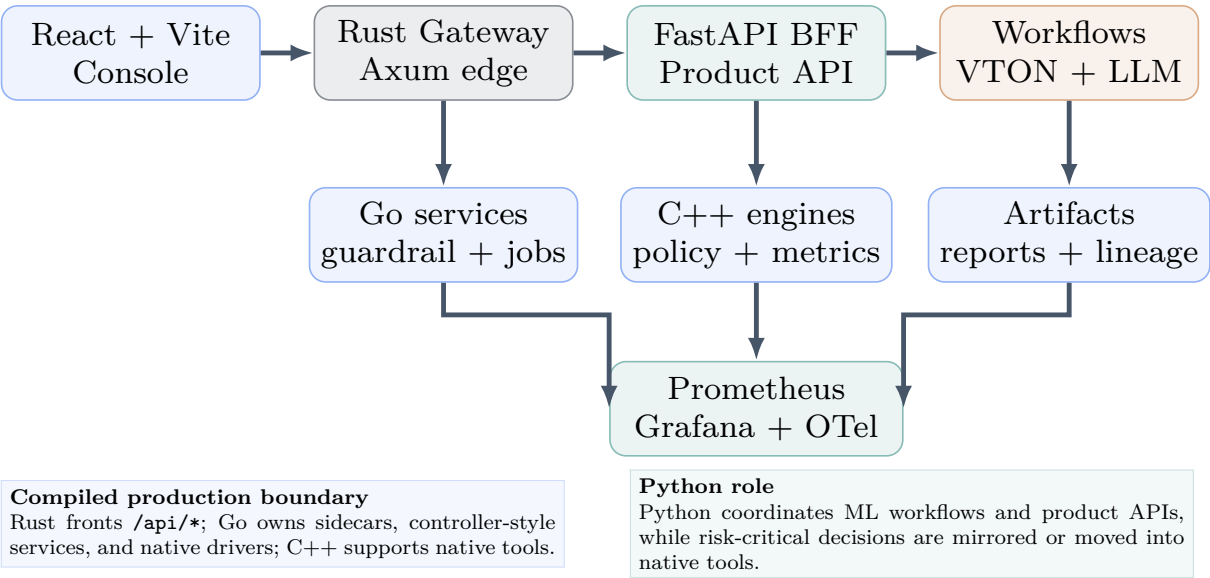


Figure 4.1: Product boundary architecture diagram.

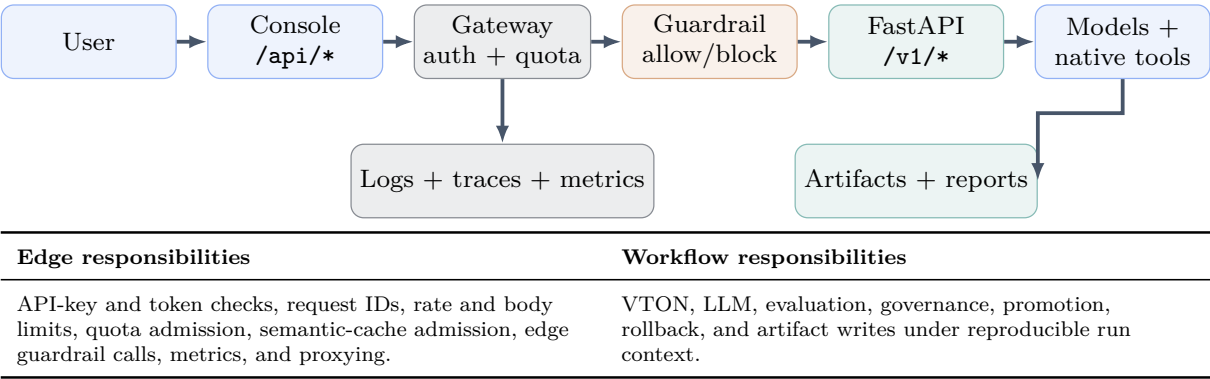


Figure 4.2: Request flow from console entry point to model output and audit records.

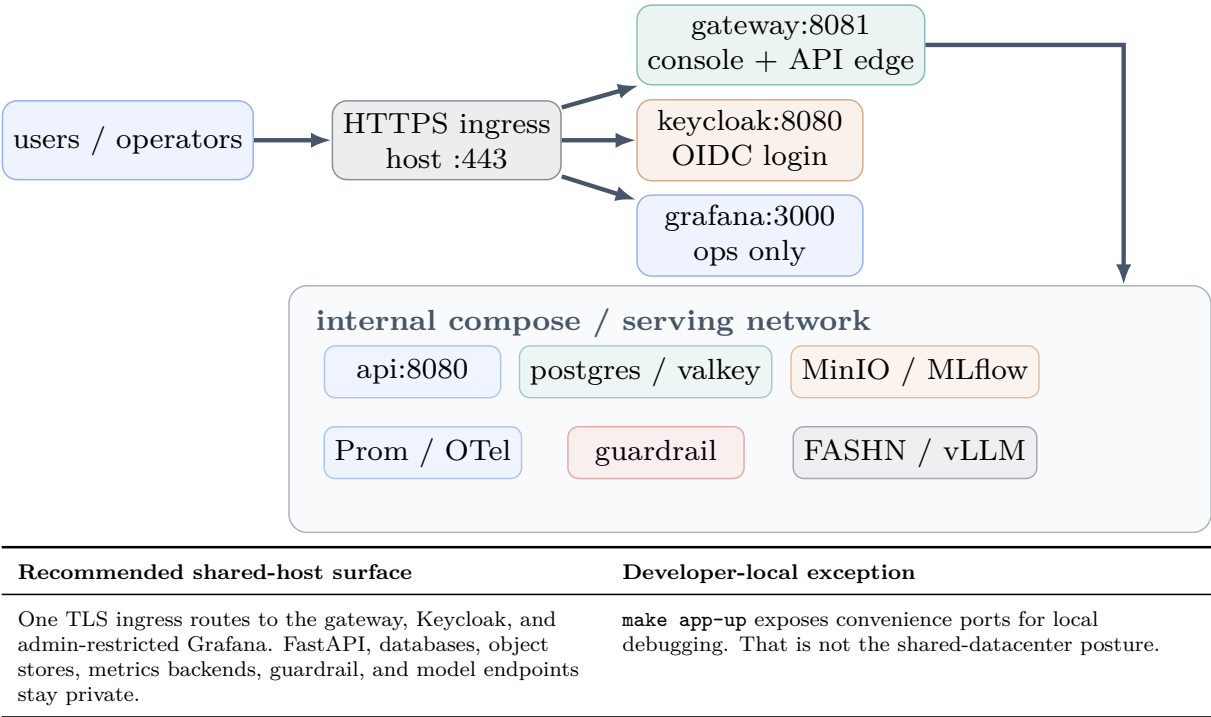


Figure 4.3: Recommended port surface for shared hosts.

Observe	Recover	Govern
OpenTelemetry trace/log correlation; Prometheus metrics and Alertmanager routing; Grafana service, quality, cost, capacity, and energy panels.	Rollback samples; chaos drills for GPU OOM, slow decode, corrupted weights, and poisoned candidates; C++ burn-rate logic for SRE-style paging.	NIST AI RMF and OWASP LLM mapping; model-source pins, SBOM, scans, SafeTensors gate; signed PR and registry-webhook release triggers.

Why it matters. The platform does not only produce model outputs; it produces operational records that an evaluator, SRE, or risk reviewer can inspect.

Figure 4.4: Observability, reliability, and governance summary.

4.2 Layered Architecture

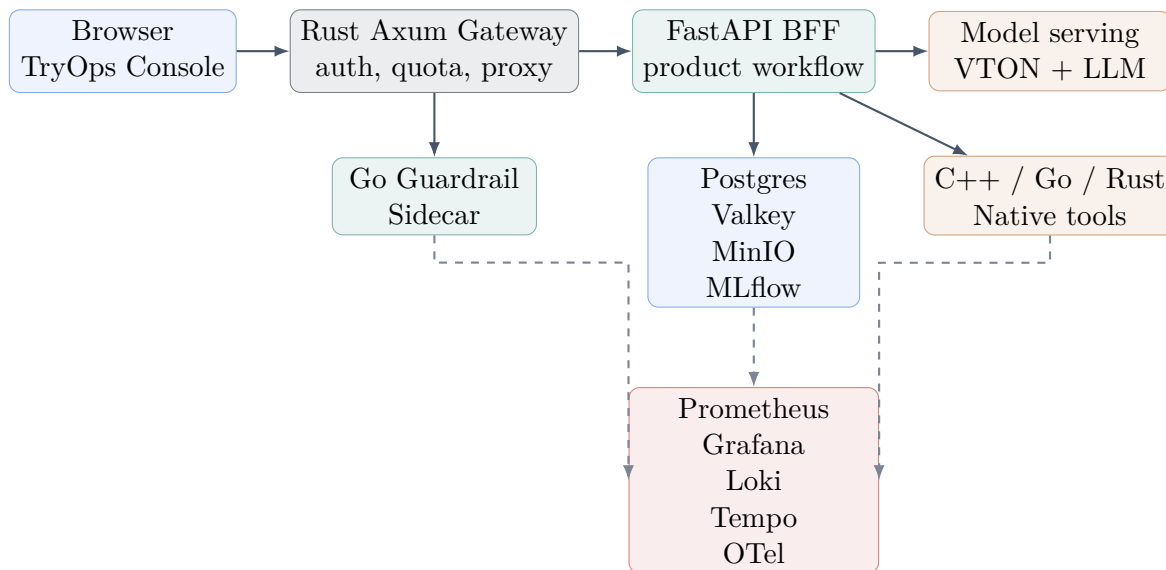


Figure 4.5: TryOps layered architecture.

4.3 Frontend Layer

The frontend is organized around product workflows rather than isolated examples. Its main capabilities include:

- VTON image upload, garment/source/quality selection, and asynchronous job submission.
- Active, queued, completed, and failed job presentation.
- Recent VTON output gallery.
- LLM prompt workflow.
- Workspace quota and account state.
- Operator views for dashboards, governance, registry state, incidents, and rollback records.
- OIDC authorization-code flow with PKCE.

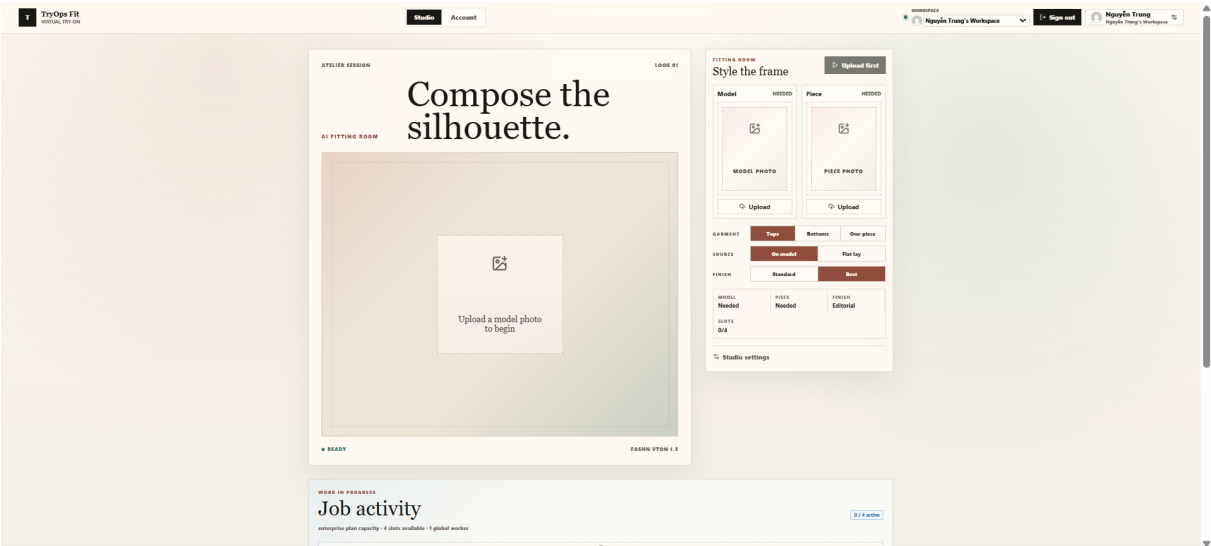


Figure 4.6: TryOps Studio product surface.

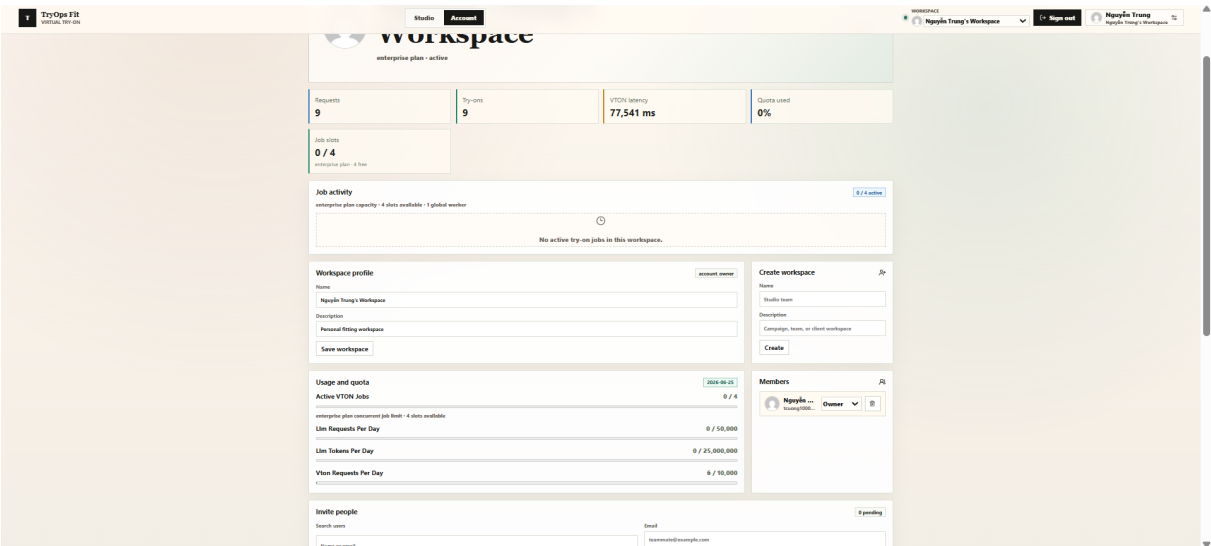


Figure 4.7: Workspace and quota view.

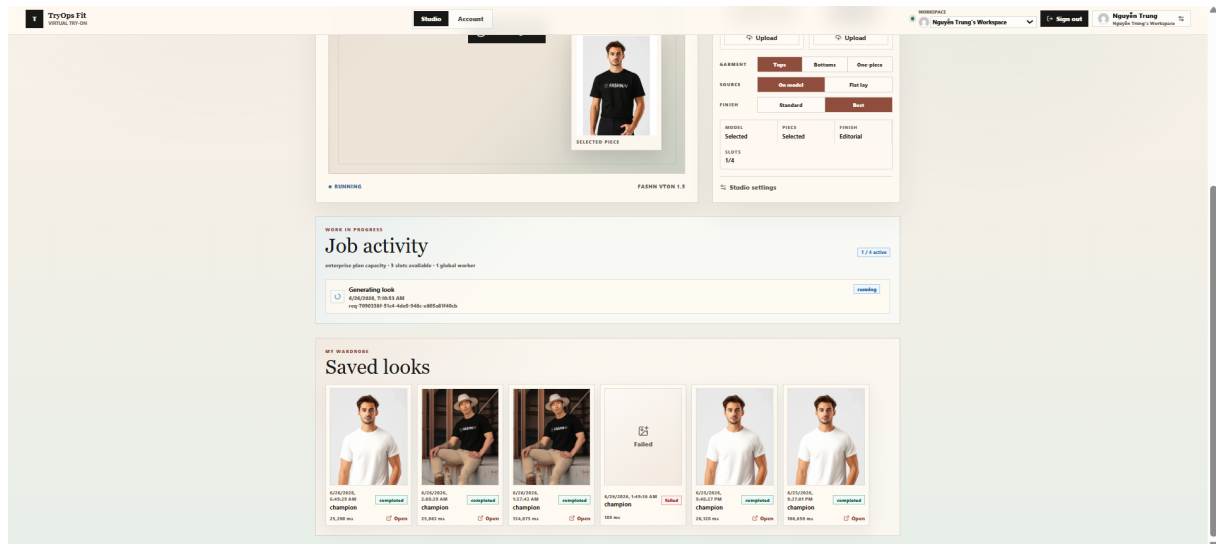


Figure 4.8: Running VTON job in the product UI.

4.4 Gateway Layer

The Rust gateway is the production-facing boundary:

- Serves static UI assets.
- Proxies product API requests to the workflow backend.
- Validates API key or JWT/OIDC identity.
- Enforces rate limit and payload limit.
- Performs quota admission before forwarding expensive work.
- Calls the Go guardrail sidecar before LLM requests.
- Forwards request IDs and trace context.
- Exposes Prometheus metrics.
- Supports TLS-oriented deployment profiles.

This design prevents Python from being the only security and quota boundary.

4.5 API and Workflow Layer

The FastAPI backend owns product workflows and coordinates:

- VTON image upload and VTON async job management.
- LLM generation.
- Account/session APIs.

- Artifact read/write.
- Dashboard rollups.
- Request history and feedback.
- Promotion, evaluation, governance, and lineage endpoints.
- Observability recording.

FastAPI does not directly own GPU process scheduling. It calls model serving through stable URLs.

4.6 Data and Artifact Layer

Component	Role
Postgres	Accounts, account members, requests, jobs, quota ledger, artifact metadata
Valkey	Hot counters for quota and rate limiting
MinIO	Object storage for uploads, outputs, and MLflow artifacts
MLflow	Experiment tracking and model registry records
DVC	Dataset and sample data versioning
Runtime artifact repository	Logs, reports, model weights, generated outputs
Generated reports repository	Promotion and evaluation reports

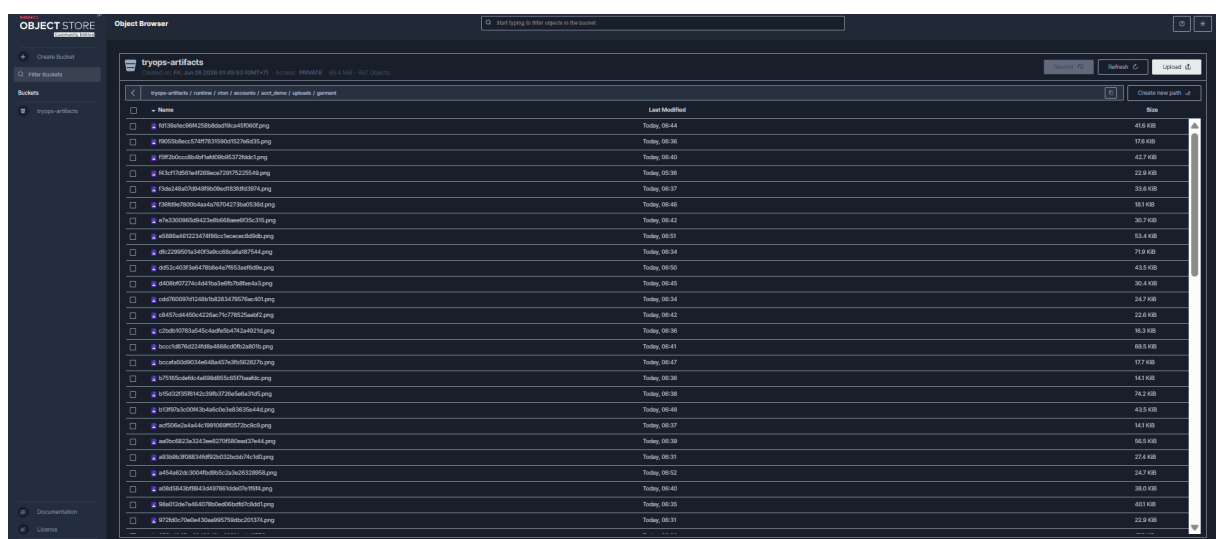


Figure 4.9: MinIO artifact store.

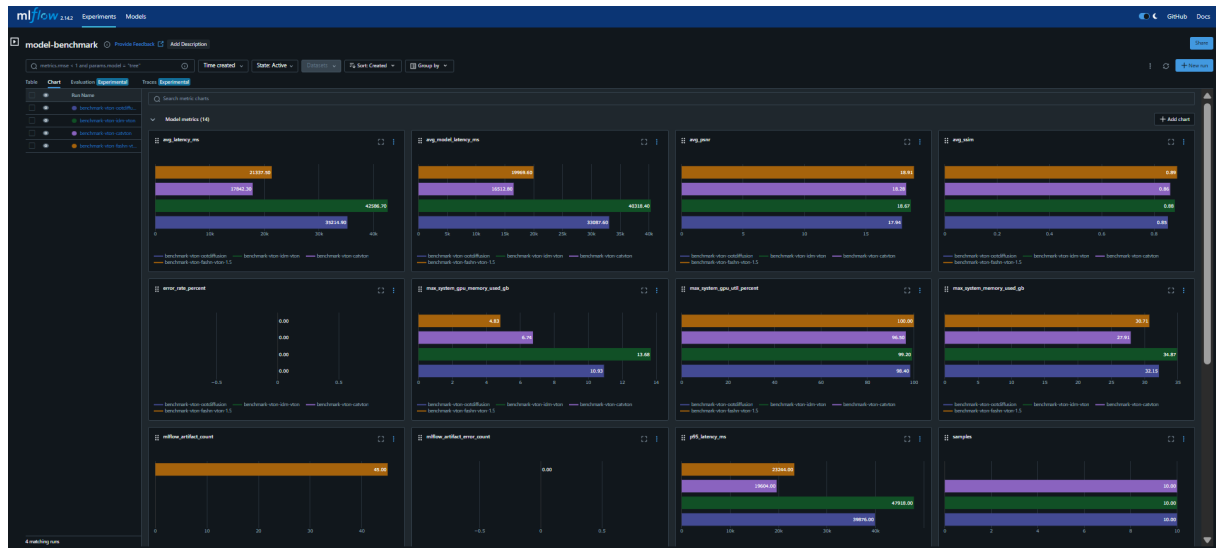


Figure 4.10: MLflow benchmark tracking.

4.7 Observability Layer

Telemetry sources include:

- FastAPI service metrics.
- Gateway service metrics.
- FASHN router and worker metrics.
- Go guardrail metrics.
- Structured API, router, and worker logs.
- Trace spans for API and background-job execution.
- OTEL Collector and bridge components for exporting logs/traces to Loki and Tempo.

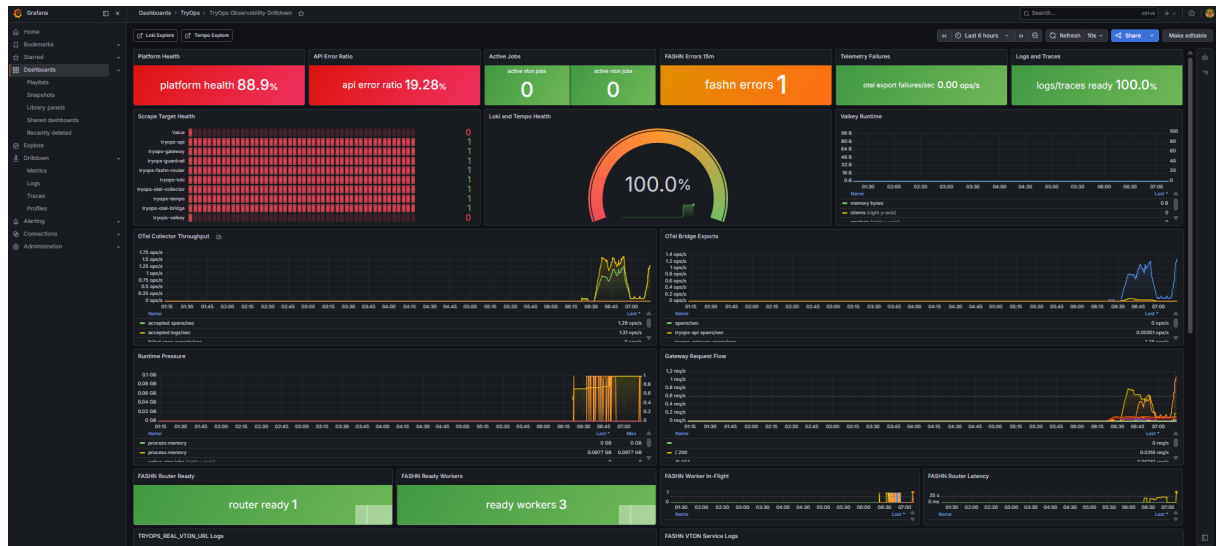


Figure 4.11: Grafana observability dashboard.

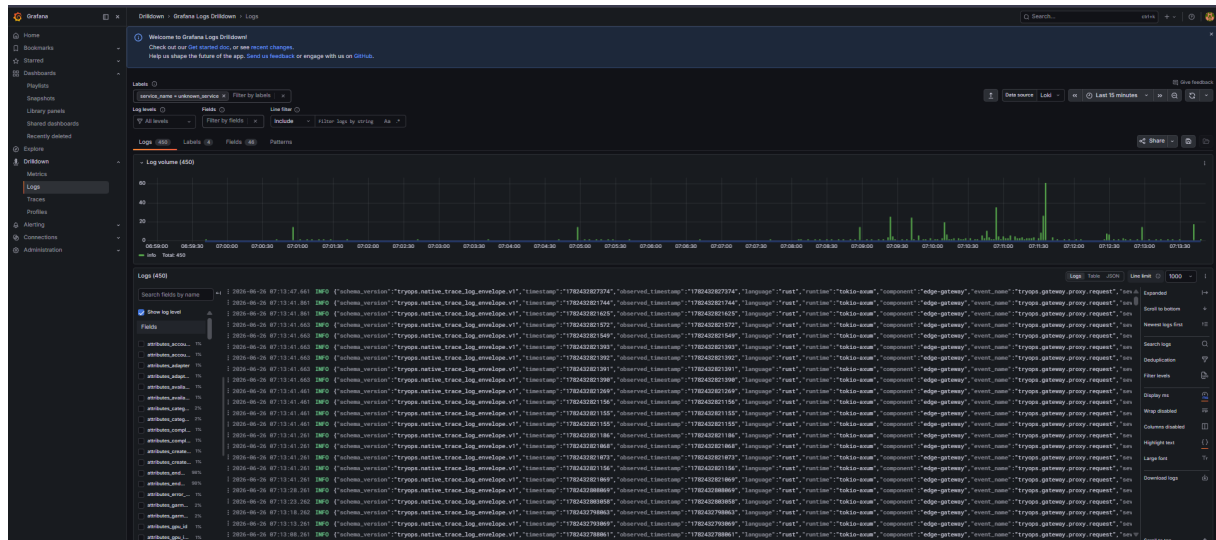


Figure 4.12: Log drilldown dashboard for debugging jobs and requests.

5. Model Serving Design

5.1 Production VTON Serving

The production-local VTON flow is organized as a layered serving path. The product API sends inference requests to a stable router endpoint. The router then dispatches work to private FASHN worker processes that are pinned to CUDA-capable GPUs. This hides GPU topology from the product API while preserving per-worker metrics for operations.

At the model level, FASHN VTON conditions generation on person pose, a clothing-agnostic person representation, garment pose, garment image, noisy image state, timestep, and garment category. These inputs are combined before the transformer blocks so the model can preserve body structure while transferring garment appearance.

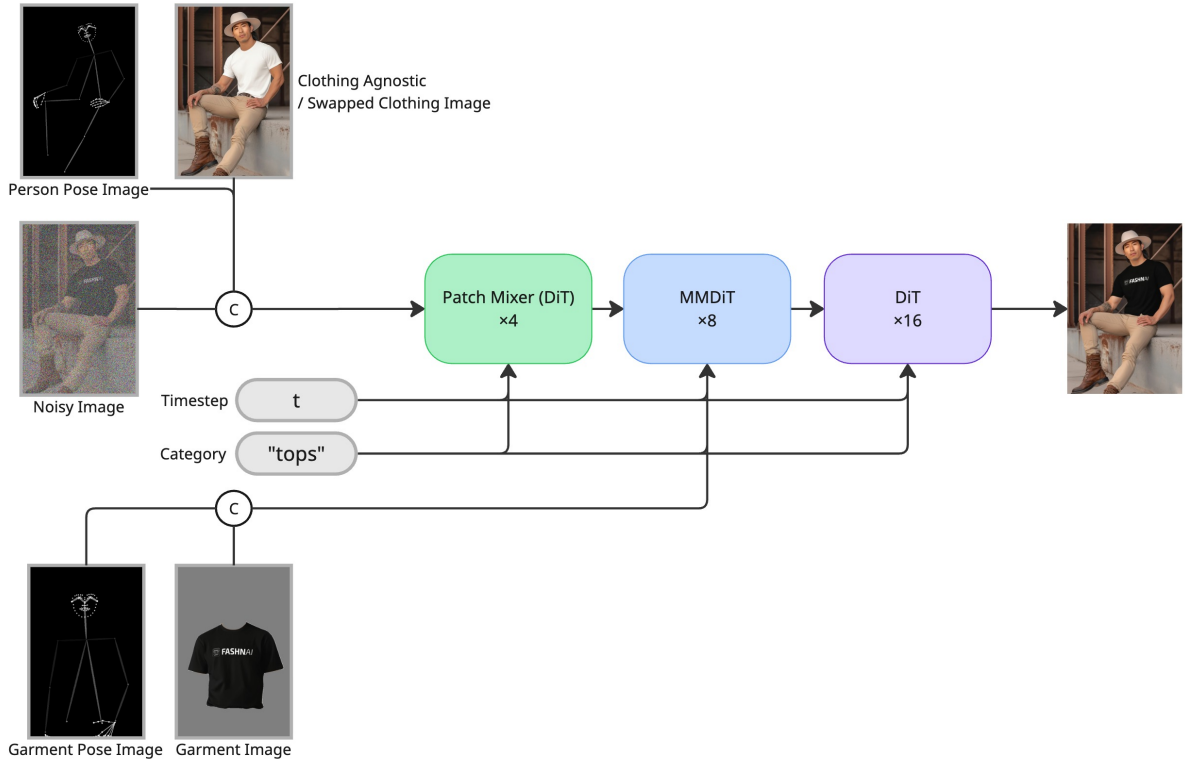


Figure 5.1: FASHN VTON v1.5 model flow. Pose and garment conditioning, timestep, and category information pass through Patch Mixer, MMDiT, and DiT blocks before producing the final try-on image. Source: FASHN VTON v1.5 research page.

The FASHN router is the abstraction boundary:

- Reads the configured GPU assignment.
- Creates one worker definition per configured GPU.
- Pins each worker to a specific CUDA device.
- Uses Unix sockets by default to avoid public worker ports.
- Performs readiness checks.
- Routes requests through least-inflight plus round-robin behavior.
- Exports Prometheus metrics with `worker_id`, `gpu_id`, `gpu_uuid`, and `status` labels.
- Writes structured router and worker logs.

5.2 VTON Job Lifecycle

1. User uploads the person image and garment image.
2. API normalizes and stores the image artifacts.
3. User submits a VTON job.
4. API checks payload, auth, quota, and workspace concurrency.
5. The VTON job queue persists a job snapshot.
6. Runner calls the VTON inference implementation.
7. Adapter calls the configured FASHN router endpoint.
8. FASHN worker generates output PNG and report metadata.
9. API persists request record and job result.
10. UI polls job status and opens the output artifact.

Status	Meaning
accepted	API accepted the job
queued	Job is waiting in the queue
running	Job is executing
completed	Job has an output artifact
failed	Job failed and has an error message

5.3 Multi-GPU Worker Configuration

In a multi-GPU deployment, the router is configured with the GPU identifiers that it may use. Each worker is assigned to one GPU and communicates over private internal endpoints. The product API continues to call only the router, while Prometheus and Grafana observe worker behavior through labels such as worker identity, GPU identity, request status, latency, and in-flight count.

Worker	CUDA pin	Private endpoint
flashn-gpu0	CUDA_VISIBLE_DEVICES=0	Unix socket
flashn-gpu1	CUDA_VISIBLE_DEVICES=1	Unix socket
flashn-gpu2	CUDA_VISIBLE_DEVICES=2	Unix socket

The API and Prometheus do not need to know the private worker endpoints. They talk to the router. Grafana can still display per-worker activity because the router emits worker and GPU labels.

5.4 Production LLM Serving

LLM serving uses an OpenAI-compatible adapter. The adapter calls the chat-completions interface and records:

- model name,
- provider/base URL after sanitization,
- input and output token counts,
- latency,
- tokens per second,
- estimated cost when configured,
- guardrail flags,
- structured output validation where requested.

When the endpoint is unreachable or returns an invalid response, the error is surfaced. That is the correct production behavior.

6. Governance, Security, and Responsible AI

6.1 Authentication and Workspace Model

TryOps uses Keycloak as the OIDC identity provider. The browser authenticates through an authorization-code flow with PKCE. The gateway validates tokens and forwards principal metadata to the backend. On first login, the backend bootstraps a workspace account in Postgres.

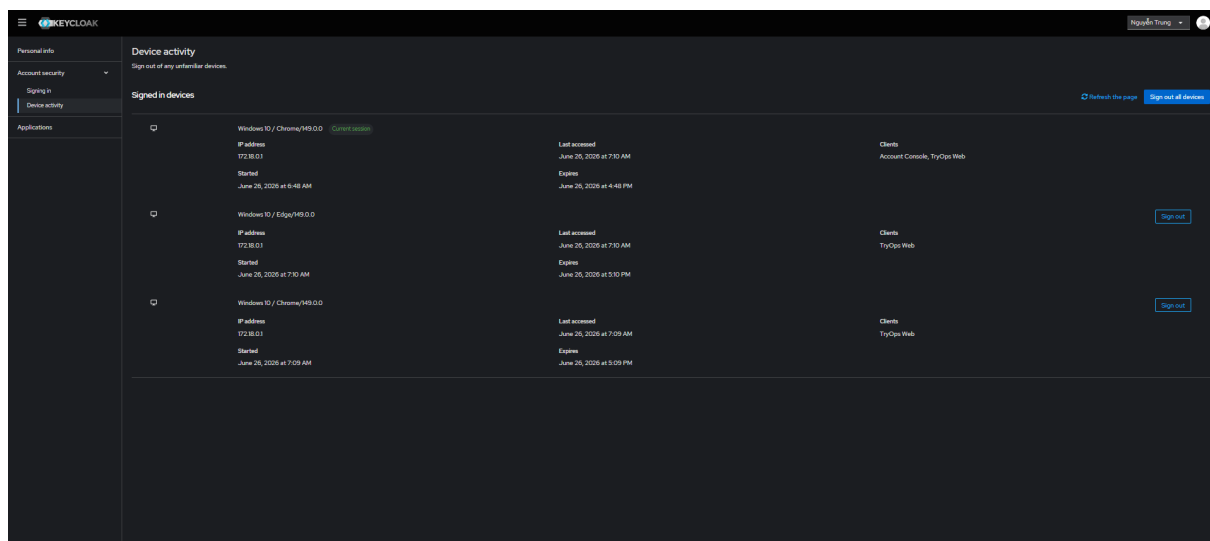


Figure 6.1: Keycloak session control and identity surface.

6.2 Quota and Admission Control

Quota is enforced at two levels:

- The Rust gateway performs edge quota/rate admission.
- The FastAPI backend enforces workload-level VTON workspace concurrency.

Plan	Active VTON jobs
free	1
team	2
enterprise	4

Plan limits are different from GPU worker count. A workspace may be limited to one active job even when the router has multiple GPU workers.

6.3 Guardrails

Guardrails run at gateway and API boundaries:

- Prompt injection detection.
- System prompt leakage prevention.
- PII and secret-like string masking.
- Output size and structured output validation.
- Metrics grouped by risk category.

The Go guardrail service is called by the gateway and API through configured internal service URLs.

6.4 Promotion Gate

Model promotion requires:

- model card,
- data card,
- evaluation report,
- SBOM,
- model artifact scan,
- provenance attestation,
- risk controls,
- owner approvals,
- policy verdict.

State	Meaning
candidate	Produced by pipeline, not trusted yet
challenger	Passed staging gate, eligible for shadow/canary
champion	Current promoted model
rejected	Blocked by policy
archived	Kept for audit and rollback history

6.5 Supply Chain and Privacy

The repository contains supply-chain and model-source documentation for model origin, dependency inventory, dataset licensing, and promotion decisions. Production logs should not contain raw prompts, raw uploaded images, or provider secrets. Logs should reference artifacts, request IDs, trace IDs, and job IDs instead.

7. Observability and Operations

7.1 Dashboards

Dashboard	Purpose
TryOps Service Overview	Request rate, error ratio, latency, memory, queue depth, gateway errors
TryOps Model Quality	Quality score, VTON latency, LLM throughput, completed inference
TryOps Cost and Capacity	Cost, quota utilization, cache hit rate, energy/carbon
TryOps Guardrails	Blocked/redacted/reviewed findings by OWASP-style risk category
TryOps Observability Drill-down	Logs, job lifecycle, FASHN service logs, OTel/Tempo/Loki health

7.2 Logs

Useful log categories:

- API request and job lifecycle events.
- FASHN router events.
- Per-worker VTON inference events.
- Model-serving process logs.
- Trace spans for API and background jobs.

Grafana should make these logs searchable by job identifier, request identifier, account identifier, worker identifier, and GPU identifier. This is necessary to debug cases where the UI is waiting but the backend has completed or failed a job.

7.3 Metrics

Important metric groups:

- HTTP request count and duration by service and status.
- VTON job lifecycle counters and active queue depth.
- FASHN router worker requests and in-flight counts.
- Per-worker latency and failure count.
- LLM token throughput, latency, and error rate.
- Guardrail block/redaction counters.
- Quota admission and remaining capacity.
- Hardware usage from host exporters and GPU exporters.

7.4 Developer-Local Ports

Figure 7.1 separates three concerns: host ports exposed during local development, private Compose-network endpoints, and host-side inference endpoints used by GPU-serving processes.

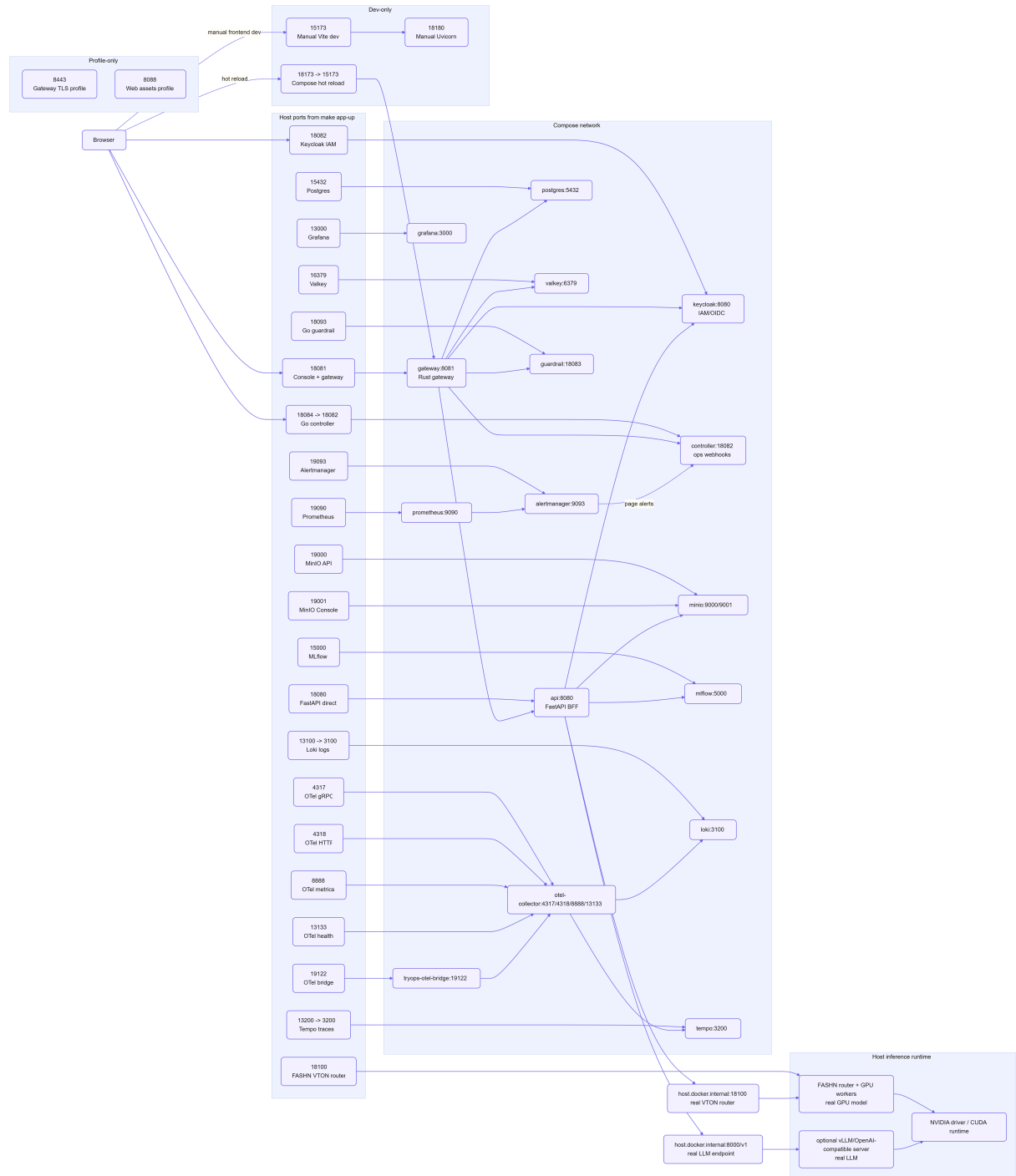


Figure 7.1: Developer-local service and port topology.

Table 7.2: Developer-local `make app-up` ports and override variables.

Service	Port	Override variable	Notes
TryOps Console + Rust gateway	18081	TRYOPS_GATEWAY_PORT	Main app URL. Use this entry point first.
FastAPI backend direct	18080	TRYOPS_API_PORT	Direct API/docs access. The gateway normally proxies this service.
Keycloak IAM	18082	TRYOPS_KEYCLOAK_PORT	OIDC/IAM service. Container port is 8080.
Go controller	18084	TRYOPS_CONTROLLER_PORT	Webhook/control-plane service. Container port is 18082; Alertmanager calls <code>controller:18082/alerts/webhook</code> inside Compose.
FASHN VTON router	18100	FASHN_VTON_ROUTER_PORT	Host-side production-local VTON router started by <code>make app-up</code> . The API reaches it through <code>host.docker.internal:18100</code> .
FASHN VTON single-worker debug	18101	FASHN_VTON_PORT	Optional direct service for isolated debugging with <code>make fashn-vton-service-bg</code> . <code>make app-up</code> does not use it.
Optional local vLLM/OpenAI-compatible LLM	8000	VLLM_PORT VLLM_BASE_URL TRYOPS_LLM_BASE_URL	Host-side LLM endpoint for self-hosted vLLM. If TRYOPS_LLM_BASE_URL points to an external OpenAI-compatible provider, no local LLM port is needed.
Postgres	15432	TRYOPS_POSTGRES_PORT	Database for the full Compose stack.
Valkey	16379	TRYOPS_VALKEY_PORT	Hot quota and rate-counter store.
MinIO API	19000	TRYOPS_MINIO_PORT	Object and artifact storage API.
MinIO Console	19001	TRYOPS_MINIO_CONSOLE_PORT	Browser UI for MinIO.
MLflow	15000	TRYOPS_MLFLOW_PORT	Experiment and model tracking.
Prometheus	19090	TRYOPS_PROMETHEUS_PORT	Metrics database.
Alertmanager	19093	TRYOPS_ALERTMANAGER_PORT	Alert routing.
Grafana	13000	TRYOPS_GRAFANA_PORT	Dashboards. Fresh local volume defaults to <code>admin / admin</code> .
Loki	13100	TRYOPS_LOKI_PORT	Log store used by Grafana when observability is enabled. Container port is 3100.
Tempo	13200	TRYOPS_TEMPO_PORT	Trace store used by Grafana when observability is enabled. Container port is 3200.
Go guardrail sidecar	18093	TRYOPS_GUARDRAIL_PORT	LLM guardrail service. Usually called by gateway/API. Container port is 18083.
OpenTelemetry gRPC	4317	TRYOPS_OTEL_GRPC_PORT	OTLP gRPC receiver.
OpenTelemetry HTTP	4318	TRYOPS_OTEL_HTTP_PORT	OTLP HTTP receiver.
OpenTelemetry metrics	8888	TRYOPS_OTEL_METRICS_PORT	Collector metrics endpoint.
OpenTelemetry health	13133	TRYOPS_OTEL_HEALTH_PORT	Collector health endpoint.
OTel bridge metrics	19122	TRYOPS_OTEL_BRIDGE_PORT	Metrics endpoint path is <code>/metrics</code> . Bridges local TryOps JSONL logs/traces into OTLP for Grafana, Loki, and Tempo.

For a shared data center, this broad developer-local port surface should be replaced by a smaller ingress surface: gateway, Keycloak, and admin-restricted Grafana. Databases, object stores, metrics stores, guardrail, and model endpoints should remain private.

7.5 Deployment Preconditions

A production-facing deployment must be configured with model endpoints, secured provider credentials, disabled development shortcuts, and explicit GPU readiness checks for VTON inference. Operational checks should verify service health, authentication, quota admission, object storage, observability, and model-serving readiness before users are allowed to submit expensive inference jobs.

8. Experiments and Evaluation

8.1 VTON Benchmark Results

Best quality score 0.888121 SSIM	Best image fidelity 18.915 PSNR	Lowest VTON memory 4.825 GB VRAM
Fastest average latency 17,842.30 ms	Highest VTON throughput 0.054261 req/s	

Table 8.1: Detailed VTON benchmark results used for model selection.

Candidate	SSIM	PSNR	Avg ms	P95 ms	req/s	Model ms	GPU %	VRAM GB
FASHN VTON v1.5	0.888121	18.915	21,337.50	23,244.00	0.045439	19,969.60	100.00	4.825
CatVTON	0.861734	18.284	17,842.30	19,604.00	0.054261	16,512.80	96.50	6.742
IDM-VTON	0.879462	18.671	42,586.70	47,918.00	0.022751	40,318.40	99.20	13.684
OOTDiffusion	0.852918	17.936	35,214.90	39,876.00	0.027502	33,087.60	98.40	10.927

Selection reading. FASHN VTON v1.5 is the strongest quality-first candidate in this run because it has the best SSIM, best PSNR, and lowest VRAM usage. CatVTON is the strongest latency-first candidate, but its recorded quality metrics are lower.

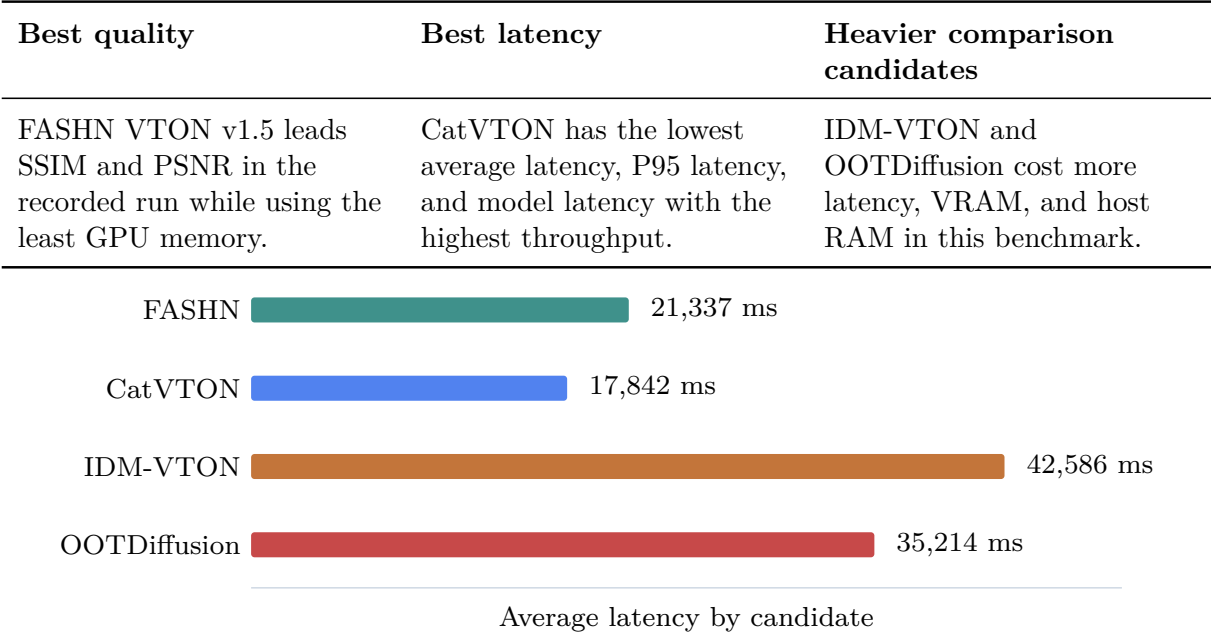


Figure 8.1: VTON benchmark interpretation and average latency comparison.

Interpretation:

- FASHN VTON v1.5 leads SSIM and PSNR in the recorded table and uses the least GPU memory.
- CatVTON has the best latency/throughput but lower recorded quality.
- IDM-VTON and OOTDiffusion consume more latency, VRAM, and host RAM.
- FASHN is a reasonable production-local target for a quality-first product path.

8.2 LLM Optimization Results

The LLM optimization summary records a Pareto frontier:

Table 8.2: Highlighted LLM optimization measurements.

Mode	VRAM	Decode speed	Reading
fp16	1.010 GB	27.4 tokens/s	Fastest measured decode speed.
8-bit	0.647 GB	5.9 tokens/s	Lower memory than fp16, but slower in this recorded run.
4-bit	0.478 GB	13.9 tokens/s	Recommended low-memory option in this comparison.

The engineering lesson is that lower precision does not automatically mean lower total cost or energy. If a quantized model decodes much slower at similar board power, joules per token and rental cost per million tokens may increase.

8.3 Native Boundary Evaluation

Benchmark results report that the Go load driver measured Rust `/health` at **24,840.8 requests/s** versus FastAPI at **1,539.4 requests/s** on the same machine, a throughput difference of about **16.1x**. This does not mean Python is removed. It means Python remains the ML workflow layer while Rust/Go/C++ support the edge, control plane, and native tooling.

8.4 Balancer Testing

The expected multi-GPU router test sends concurrent requests to the router and verifies that completed-request counters increase across multiple worker and GPU labels. If only one GPU receives all work, the first things to check are workspace job concurrency, API job worker count, router worker readiness, GPU assignment, and whether the test sends enough simultaneous jobs to exceed one worker's in-flight capacity.

9. Limitations and Future Work

9.1 Current Limitations

- Full vLLM live benchmark depends on the active host environment and installed serving runtime.
- Production KServe/Kubeflow cluster deployment is a target profile, not the currently packaged local stack.
- Representative human evaluation panels for VTON quality and fairness remain future work.
- Over-quantizing the VTON model can produce blurred garments, lost texture, and visually invalid try-on outputs; quantized VTON deployment must be gated by image-quality regression tests instead of only API success checks.
- Some UI operator workflows need additional click-through detail and export features.
- Shared-datacenter port reduction should be completed by moving internal services behind ingress/private networks.
- GPU utilization dashboards require host GPU exporters and per-worker router metrics to be consistently scraped.
- Active VTON jobs must remain visible after browser refresh while queued or running; this is essential for trustworthy handling of expensive GPU work.

One concrete VTON limitation observed during optimization is that aggressive quantization or overly compressed runtime settings can preserve API success while reducing visual quality. The output may become blurred around garment and body regions, with weak texture transfer and degraded person boundaries. This means VTON promotion cannot rely only on HTTP success, latency, and GPU memory; it also needs image-quality regression checks.

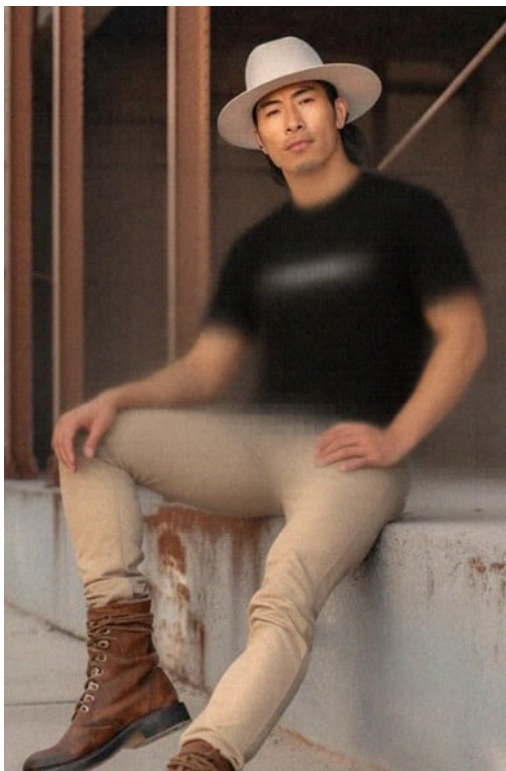


Figure 9.1: VTON over-quantization limitation: reduced runtime cost can produce blurred garment and body regions, so optimized variants require visual quality gates before promotion.

9.2 Future Work

1. Finish active job persistence and refresh-safe UI behavior.
2. Harden shared-host deployment with a minimal public port surface.
3. Add host GPU exporter integration and Grafana GPU worker dashboards.
4. Add a production secret manager integration.
5. Expand VTON evaluation with human preference labels and fairness slices.
6. Add broader LLM provider/model compatibility tests.
7. Package a KServe/Kubeflow profile for cluster-grade deployment.
8. Improve incident response runbooks and dashboard drilldowns.

9.3 Implementation Boundary

The strongest position for this project is to be explicit:

- Implemented paths: Rust gateway, FastAPI backend, Keycloak, MinIO, MLflow, Prometheus/Grafana/Loki/Tempo, FASHN VTON router, OpenAI-compatible LLM

adapter.

- Local/conditional paths: vLLM live benchmark and GPU exporter availability depend on host runtime.
- Test/evaluation paths: seeded examples and offline evaluation fixtures.
- Future paths: full cluster deployment, larger fairness panels, production-grade external secret handling.

This separation makes the project more defensible, not less. It prevents overclaiming and gives operators a clear list of what is production-ready versus what still needs hardening.

10. Conclusion

TryOps demonstrates an MLOps platform that treats models as governed production assets rather than isolated notebook outputs. The project integrates a usable product console, model-serving boundaries, account and quota control, artifact storage, model governance, observability, native control-plane components, and explicit production readiness checks.

The most important result is architectural: VTON and LLM workloads can share the same operational spine while preserving workload-specific needs. VTON requires GPU-aware job control, image artifact lineage, and visual quality assessment. LLM serving requires token accounting, endpoint health, guardrails, and latency/cost measurements. TryOps uses the same gateway, API, observability, storage, and governance foundation for both.

The project still has work to do before it can be treated as a hardened multi-tenant data-center platform. However, its current design and repository artifacts already show the core MLOps contribution: production model paths are separated from diagnostics, model changes must earn promotion, and operators can inspect the records needed to debug, govern, and roll back the system.

References

Bibliography

- [1] OpenTelemetry, “OpenTelemetry Documentation,” [Online]. Available: <https://opentelemetry.io/docs/>. Accessed: July 4, 2026.
- [2] Prometheus Authors, “Prometheus Documentation,” [Online]. Available: <https://prometheus.io/docs/>. Accessed: July 4, 2026.
- [3] Grafana Labs, “Grafana Documentation,” [Online]. Available: <https://grafana.com/docs/grafana/latest/>. Accessed: July 4, 2026.
- [4] MLflow, “MLflow Documentation,” [Online]. Available: <https://mlflow.org/docs/latest/>. Accessed: July 4, 2026.
- [5] MinIO, “MinIO Object Storage Documentation,” [Online]. Available: <https://min.io/docs/minio/linux/>. Accessed: July 4, 2026.
- [6] Keycloak, “Keycloak Documentation,” [Online]. Available: <https://www.keycloak.org/documentation>. Accessed: July 4, 2026.
- [7] FASHN, “FASHN VTON 1.5 Research,” [Online]. Available: <https://fashn.ai/research/vton-1-5>. Accessed: July 4, 2026.
- [8] vLLM Project, “vLLM Documentation,” [Online]. Available: <https://docs.vllm.ai/>. Accessed: July 4, 2026.
- [9] National Institute of Standards and Technology, “Artificial Intelligence Risk Management Framework,” [Online]. Available: <https://www.nist.gov/itl/ai-risk-management-framework>. Accessed: July 4, 2026.
- [10] OWASP Foundation, “OWASP Top 10 for Large Language Model Applications,” [Online]. Available: <https://owasp.org/www-project-top-10-for-large-language-model-applications/>. Accessed: July 4, 2026.
- [11] OpenSSF, “Supply-chain Levels for Software Artifacts,” [Online]. Available: <https://slsa.dev/>. Accessed: July 4, 2026.
- [12] Anchore, “Syft: SBOM Generator,” [Online]. Available: <https://github.com/anchore/syft>. Accessed: July 4, 2026.
- [13] Aqua Security, “Trivy Documentation,” [Online]. Available: <https://aquasecurity.github.io/trivy/>. Accessed: July 4, 2026.

- [14] Sigstore, “Cosign Documentation,” [Online]. Available: <https://docs.sigstore.dev/cosign/>. Accessed: July 4, 2026.